Lecture 19

Convolutional Neural Network (Cont.) and Autoencoders

Introduction to convolutional architecture for image analysis

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi

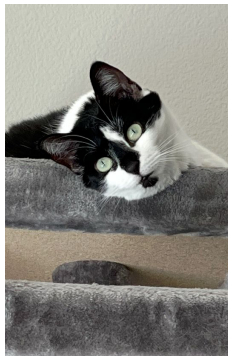
Roadmap

- Recap of CNN Design
- CNN History
- Transfer Learning
- Autoencoders

3970279

Recap of CNN Design

- Recap of CNN Design
- CNN History
- Transfer Learning
- Autoencoders



Convolution

Pooling

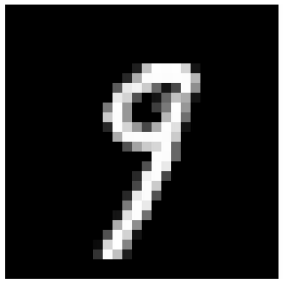
Convolution

Pooling

Flatten

Output

3970279



$1 \times 28 \times 28$

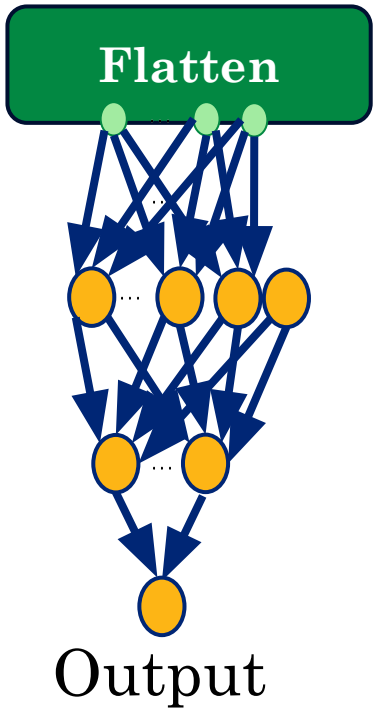
8 kernels of $1 \times 3 \times 3$

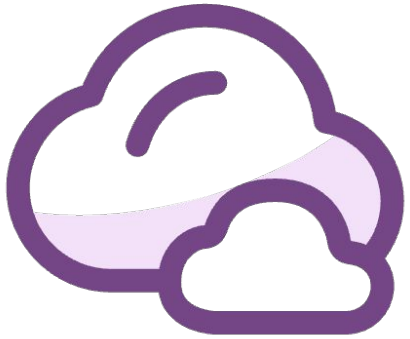
Convolution → Pooling → Convolution → Pooling

$8 \times 28 \times 28$

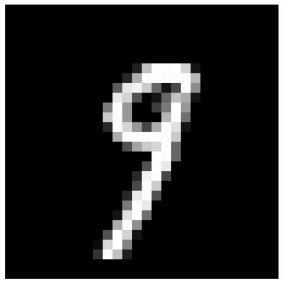
```
def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(3, 8, kernel_size=3, padding=1)
```

```
nn.Conv2d(
    in_channels,  # # of input feature maps (e.g., 3 for RGB)
    out_channels, # # of filters you learn → # of output feature maps
    kernel_size,  # filter size (int or (kh, kw))
    stride=1,     # step size of the filter (int or (sh, sw))
    padding=0,    # add zeros around the image (int, tuple, or 'same')
    bias=True,    # learn an additive bias per output channel
    padding_mode='zeros' # 'zeros' (default), 'reflect', 'replicate'
)
```





How many trainable parameters are in the first convolutional layer?



$1 \times 28 \times 28$

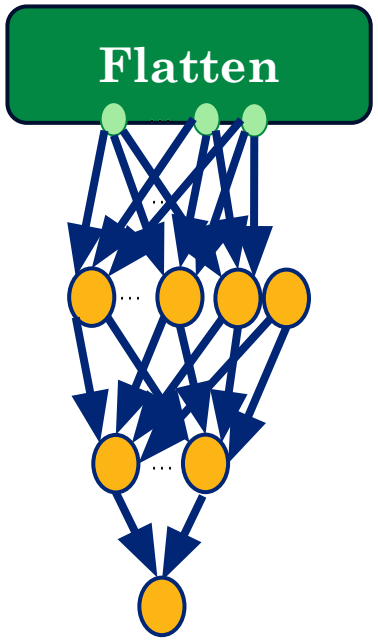
8 kernels of $1 \times 3 \times 3$

Convolution → Pooling → Convolution → Pooling → Flatten → Output

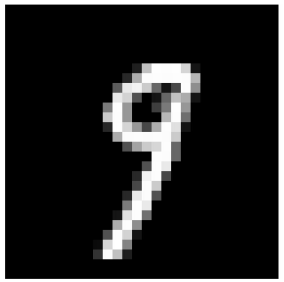
$8 \times 28 \times 28$

$8 \times 14 \times 14$

```
def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(3, 8, kernel_size=3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)
```



Output



$1 \times 28 \times 28$

8 kernels of $1 \times 3 \times 3$

Convolution

$8 \times 28 \times 28$

Pooling

$8 \times 14 \times 14$

64 kernels of $8 \times 3 \times 3$

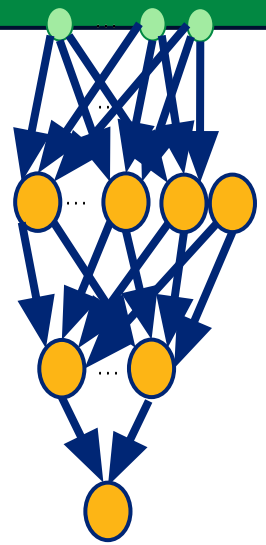
Convolution

$64 \times 14 \times 14$

Pooling

$64 \times 7 \times 7$

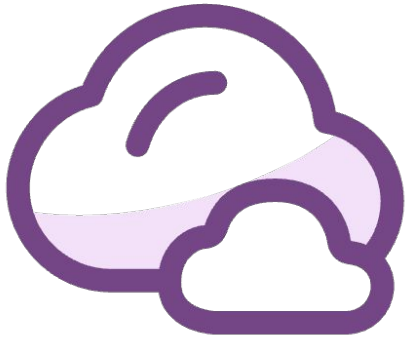
Flatten



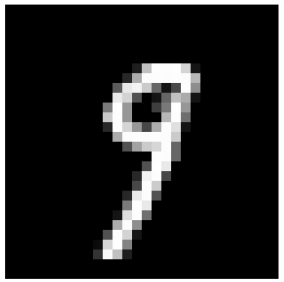
Output

```
def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(3, 8, kernel_size=3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(8, 64, kernel_size=3, padding=1)
```

3970279



How many trainable parameters are in the second convolutional layer?



$1 \times 28 \times 28$

8 kernels of $1 \times 3 \times 3$

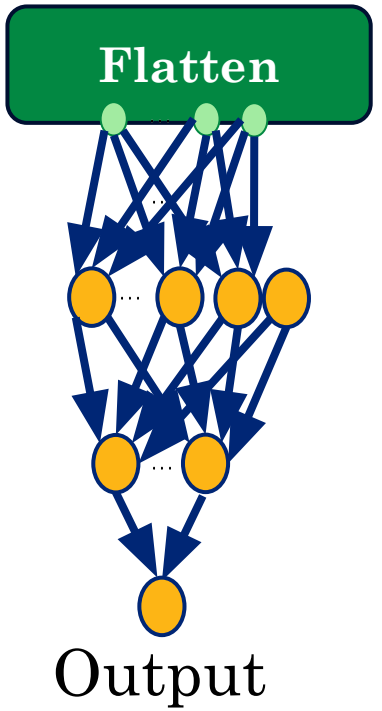
64 kernels of $8 \times 3 \times 3$

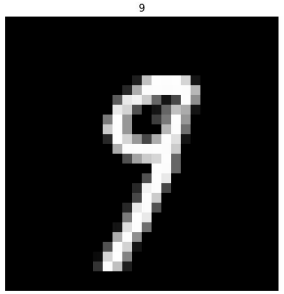
3970279

Convolution → Pooling → Convolution → Pooling → $64 \times 7 \times 7$

$8 \times 28 \times 28$ $8 \times 14 \times 14$ $64 \times 14 \times 14$

```
def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(3, 8, kernel_size=3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(8, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 7 * 7, 256)

        self.fc2 = nn.Linear(256, num_classes)


        self.dropout = nn.Dropout(0.25)
```





$1 \times 28 \times 28$

8 kernels of $1 \times 3 \times 3$

Convolution

$8 \times 28 \times 28$

Pooling

$8 \times 14 \times 14$

64 kernels of $8 \times 3 \times 3$

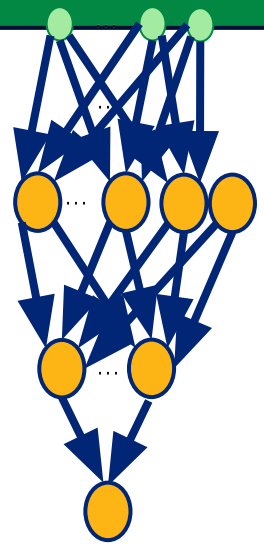
Convolution

$64 \times 14 \times 14$

Pooling

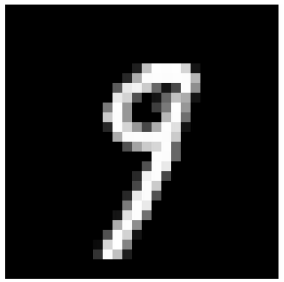
$64 \times 7 \times 7$

Flatten



Output

```
def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))    # 8x14x14
        x = self.pool(F.relu(self.conv2(x)))    # 64x7x7
        x = x.view(x.size(0), -1)                  # flatten: 64*7*7
        x = self.dropout(F.relu(self.fc1(x)))
    return self.fc2(x)
```



$1 \times 28 \times 28$

8 kernels of $1 \times 3 \times 3$

64 kernels of $8 \times 3 \times 3$

3970279

Convolution → Pooling → Convolution → Pooling

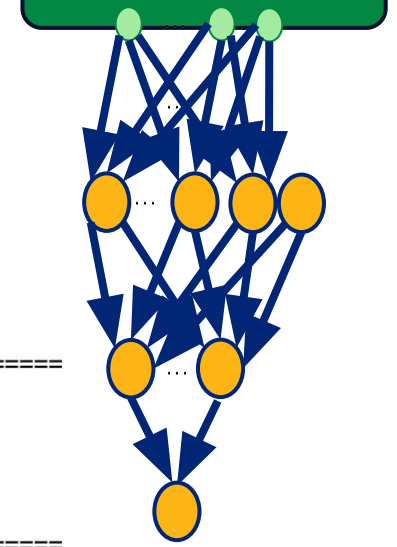
$8 \times 28 \times 28$ $8 \times 14 \times 14$ $64 \times 14 \times 14$

$64 \times 7 \times 7$

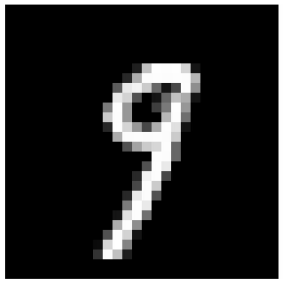
Layer (type:depth-idx)	Input Shape	Output Shape	Param #	Kernel Shape
SmallCNN	[1, 1, 28, 28]	[1, 10]	--	--
├─Sequential: 1-1	[1, 1, 28, 28]	[1, 64, 7, 7]	--	--
│ └─Conv2d: 2-1	[1, 1, 28, 28]	[1, 8, 28, 28]	80	[3, 3]
│ └─ReLU: 2-2	[1, 8, 28, 28]	[1, 8, 28, 28]	--	--
│ └─MaxPool2d: 2-3	[1, 8, 28, 28]	[1, 8, 14, 14]	--	2
│ └─Conv2d: 2-4	[1, 8, 14, 14]	[1, 64, 14, 14]	4,672	[3, 3]
│ └─ReLU: 2-5	[1, 64, 14, 14]	[1, 64, 14, 14]	--	--
│ └─MaxPool2d: 2-6	[1, 64, 14, 14]	[1, 64, 7, 7]	--	2
├─Sequential: 1-2	[1, 64, 7, 7]	[1, 10]	--	--
│ └─Flatten: 2-7	[1, 64, 7, 7]	[1, 3136]	--	--
│ └─Linear: 2-8	[1, 3136]	[1, 256]	803,072	--
│ └─ReLU: 2-9	[1, 256]	[1, 256]	--	--
│ └─Dropout: 2-10	[1, 256]	[1, 256]	--	--
│ └─Linear: 2-11	[1, 256]	[1, 10]	2,570	--

Total params: 810,394
Trainable params: 810,394
Non-trainable params: 0
Total mult-adds (M): 1.78

Flatten



Output



$1 \times 28 \times 28$

8 kernels of $1 \times 3 \times 3$

Convolution

$8 \times 28 \times 28$

Pooling

$8 \times 14 \times 14$

64 kernels of $8 \times 3 \times 3$

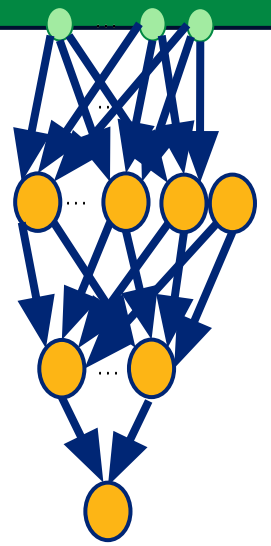
Convolution

$64 \times 14 \times 14$

Pooling

$64 \times 7 \times 7$

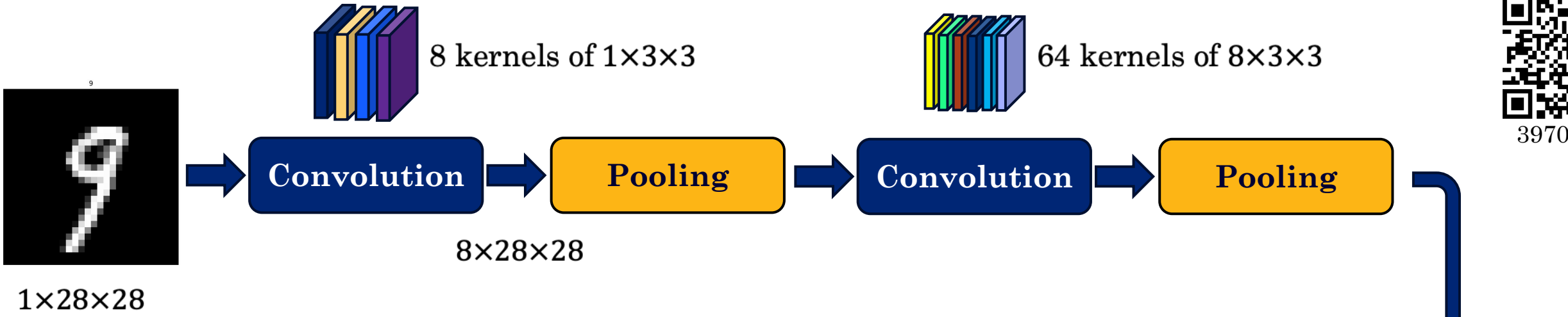
Flatten



Output

```
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), )
        self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(64*7*7, 256),
        nn.ReLU(),
        nn.Dropout(0.25),
        nn.Linear(256, 10))
    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

3970279

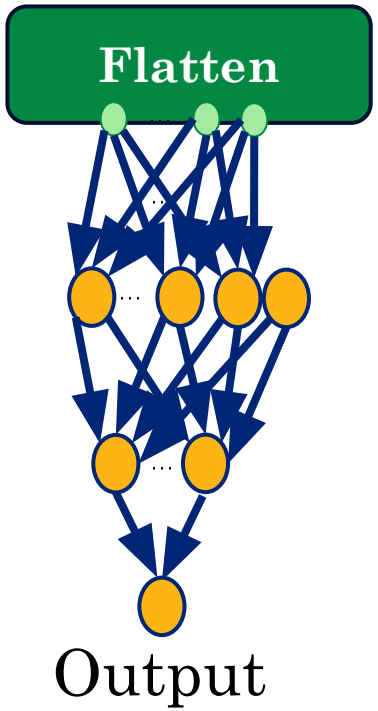


The output of the k-th filter is a 28 ×28 matrix.

Degree of the activation of the k-th filter: $a^k = \sum_{i=1}^{28} \sum_{j=1}^{28} a_{ij}^k$

$$x^* = arg \max_x a^k$$ (gradient ascent)

Find an image maximizing the output of neuron

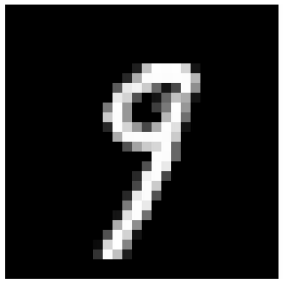


What Does CNN Learn?

8 kernels of 1×3×3

64 kernels of 8×3×3

3970279



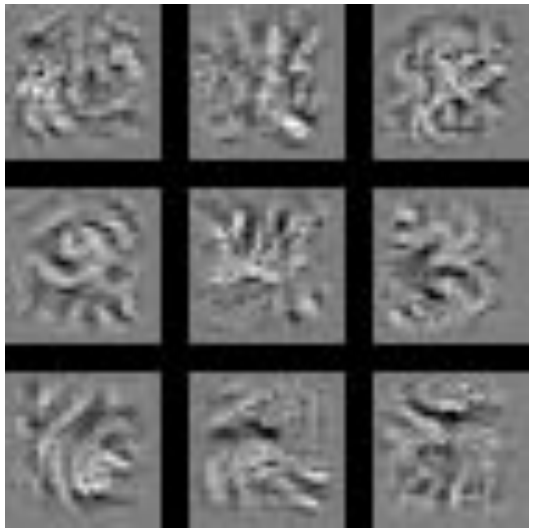
1×28×28

Convolution → Pooling → Convolution → Pooling → Flatten → Output

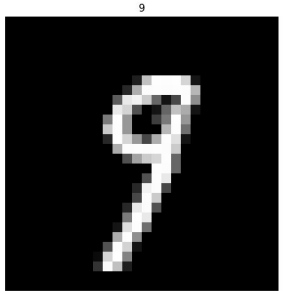
8×28×28

Degree of the activation of the k-th filter: $a^k = \sum_{i=1}^{28} \sum_{j=1}^{28} a_{ij}^k$

$x^* = arg\max_x a^k$ (gradient ascent)



What Does CNN Learn?



1×28×28

8 kernels of 1×3×3

Convolution

Pooling

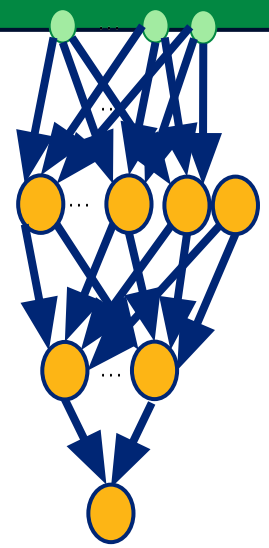
8×28×28

64 kernels of 8×3×3

Convolution

Pooling

Flatten



Output

$$x^* = arg\max_x y^c$$

What Does A Class Look Like?

Karen Simonyan, Andrea Vedaldi, Andrew Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”, ICLR, 2014

Gradient Saliency

y^c: the predicted class of the model

$$\left|\frac{\partial y^c}{\partial x_{ij}}\right|$$

Karen Simonyan, Andrea Vedaldi, Andrew Zisserman, “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”, ICLR, 2014

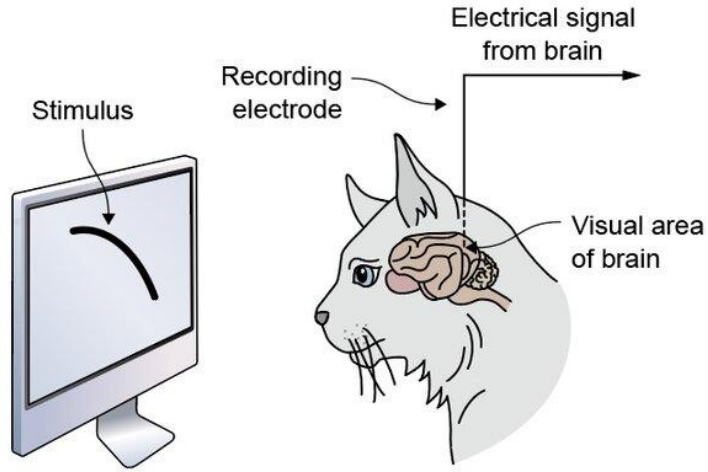
CNN History

- Recap of CNN Design
- CNN History
- Transfer Learning
- Autoencoders

History

3970279

- How do animals see?
  - What is the neural process from eye to recognition?
- Hubel and Wiesel 1959
  - First study on neural correlates of vision.
  - Receptive Fields in Cat's visual cortex
  - 24 cats, anaesthetized, immobilized, on artificial respirators
    - Electrodes in brain
  - Beamed light of different patterns into the eyes and measured neural responses



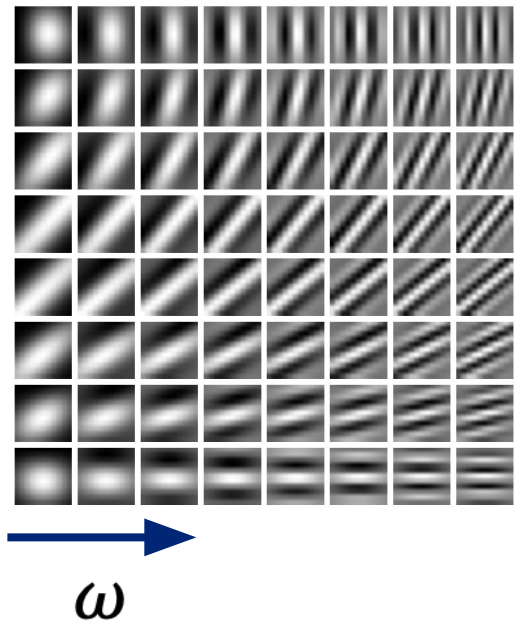
Gabor Filters

- Hubel and Wiesel study introduced two types of cells in brain:
  - **Simple cells**: Strong response to visual inputs with a simple edge oriented at a particular angle and located at a particular position.
    - More studies showed that the responses from simple cells can be modeled as Gabor filters:

$$G(x, y) = A \exp(-\alpha \tilde{x}^2 - \beta \tilde{y}^2) \sin(\omega \tilde{x} + \phi)$$

Decay envelope to localize the filter around (x_0, y_0)

$$\tilde{x} = (x - x_0)\cos\theta + (y - y_0)\sin\theta$$
$$\tilde{y} = -(x - x_0)\sin\theta + (y - y_0)\cos\theta$$



  - **Complex cells**: Respond to more complex stimuli derived by combining and processing the output of simple cells.

1980s and 1990s …

3970279

- By the end of the 1980's, several papers were produced that considerably advanced the field.
- The idea of backpropagation was first published in French by Yann LeCun in 1985.
- November 1998, LeCun published one of his most recognized papers describing a "modern" CNN architecture for document recognition, called LeNet-5.

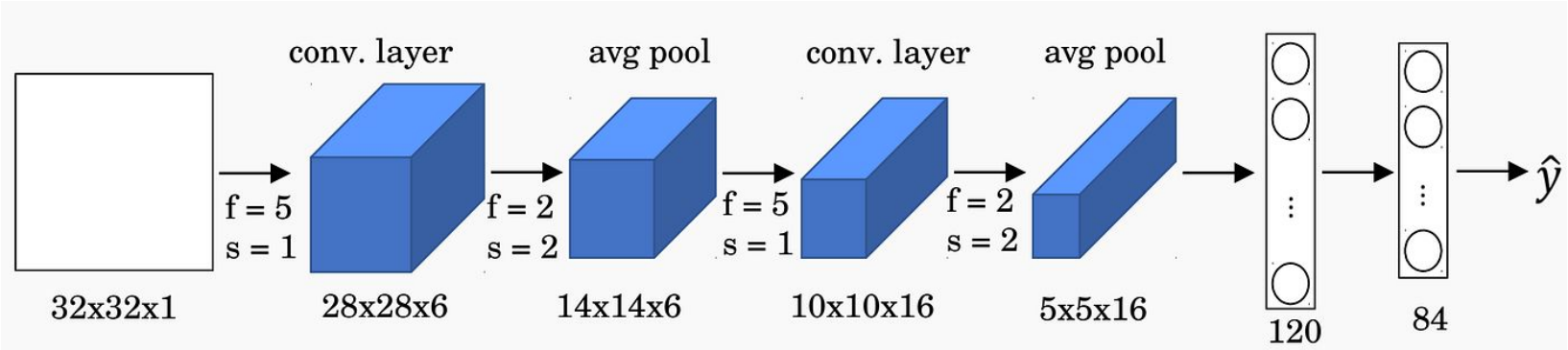


Image from https://medium.com/

LeCun, Yann, et al. "Gradient-based learning applied to document recognition." Proceedings of the IEEE 86.11 (1998): 2278–2324.

The ImageNet Task (2009)

3970279

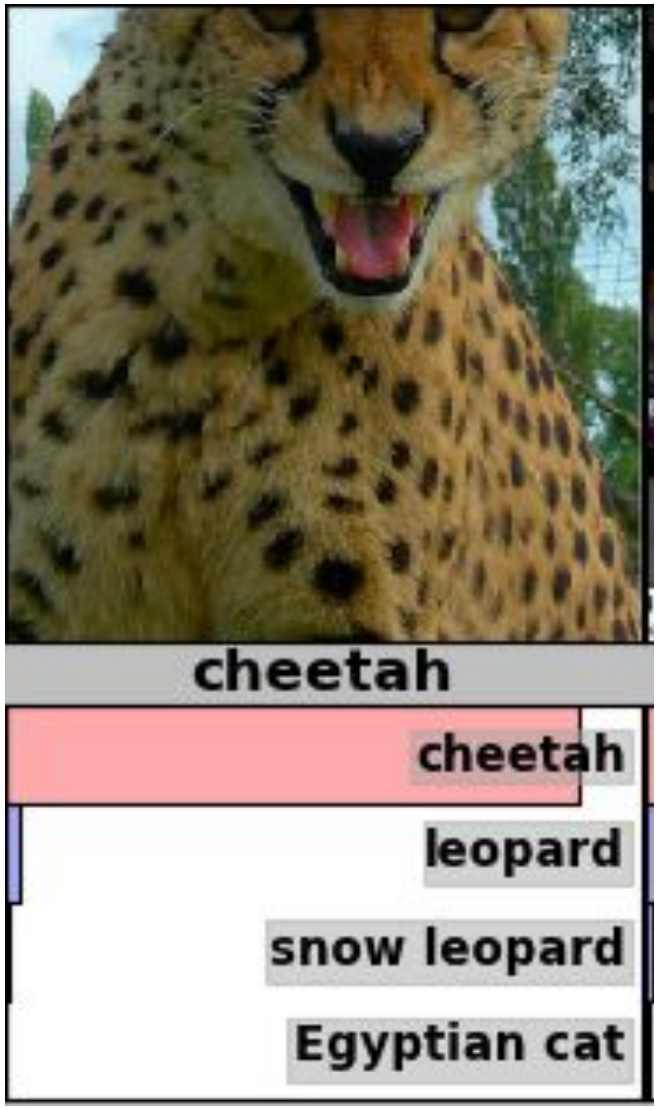
- 14 million images in more than 20,000 hierarchical categories with hand-annotation
  - Indicate what objects are pictured
  - In at least one million of the images, bounding boxes are also provided

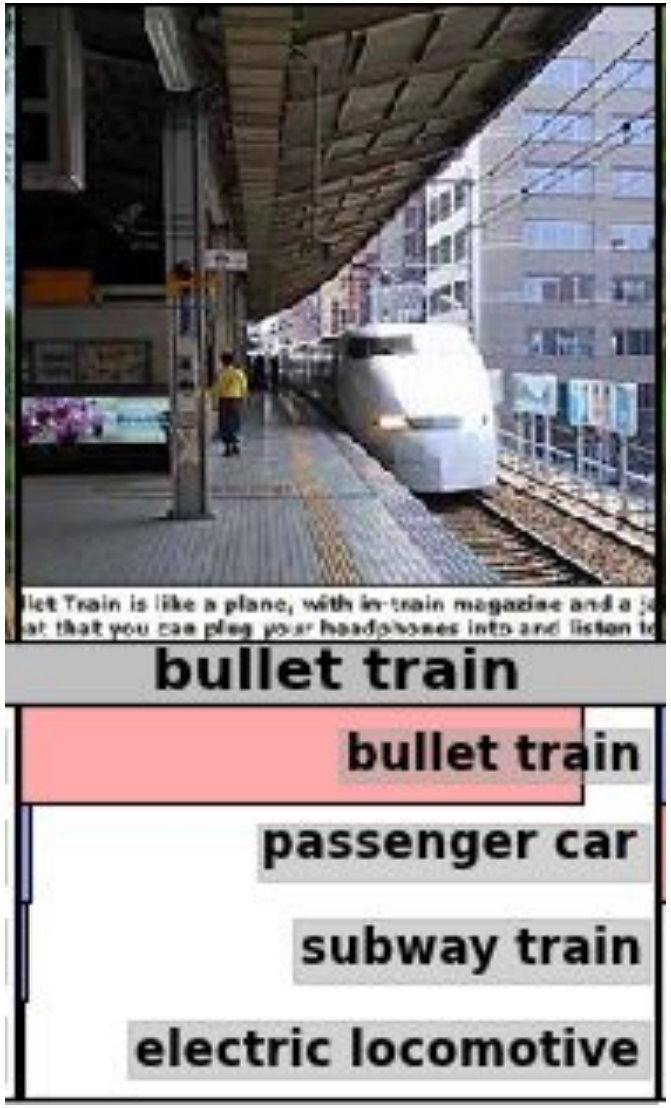
ImageNet Larg-Scale Visual Recognition Challenge (ILSVRC) (2010)

3970279

- ILSVRC evaluates algorithms for object detection and image classification at large scale.
- **The classification task:**
  - Get the "correct" class in your top 5 bets. There are 1000 classes.
- **The localization task:**
  - For each bet, put a box around the object. Your box must have at least 50% overlap with the correct box.

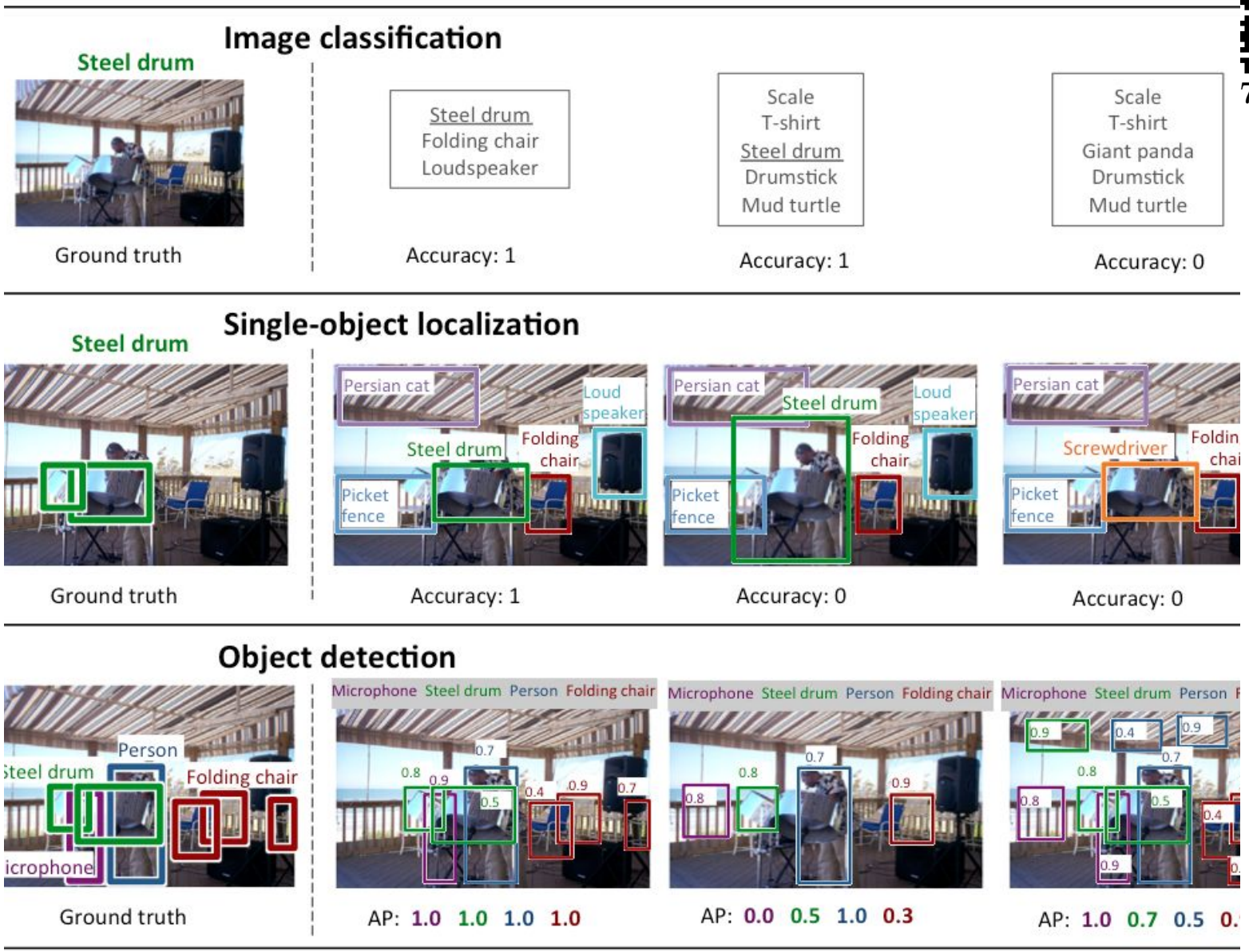
Examples from the Test Set





ImageNet Larg-Scale Visual Recognition Challenge (ILSVRC)

The first column shows the ground truth labeling on an example image, and the next three show three sample outputs with the corresponding evaluation score.

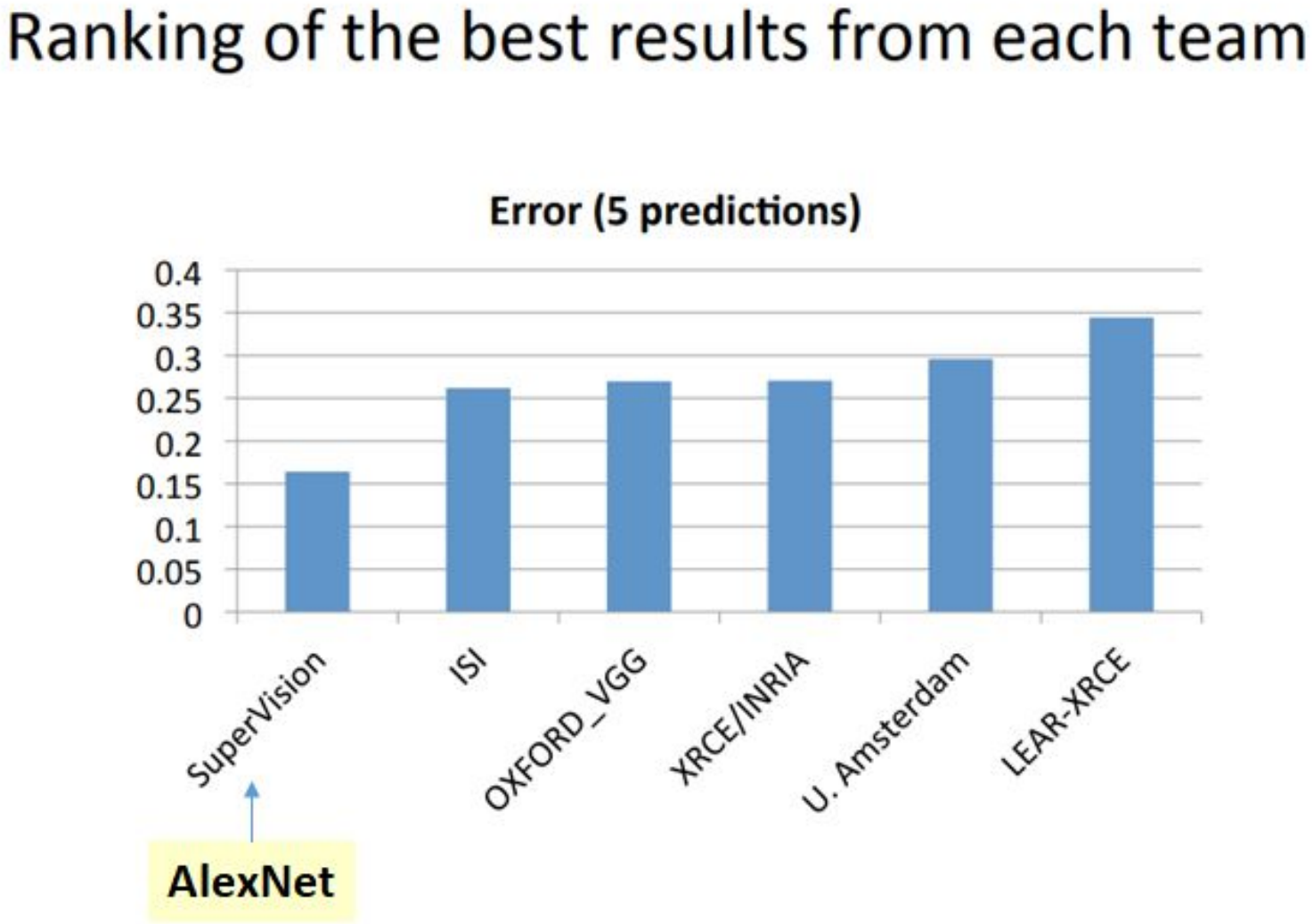


Credit: https://ar5iv.labs.arxiv.org/html/1409.0575

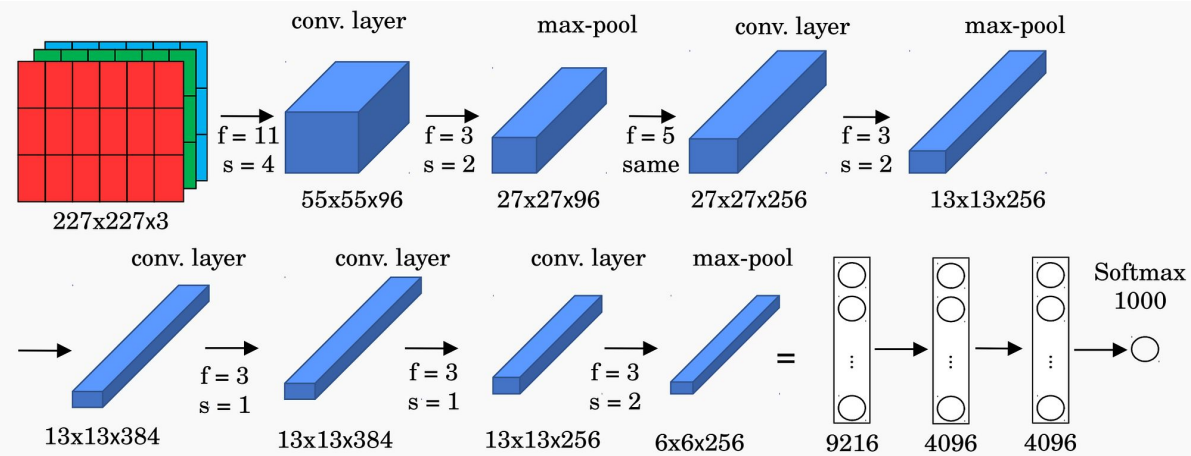
3970279

The ILSVRC-2012 Competition on ImageNet

Some of the best existing computer vision methods were tried on this dataset by leading computer vision groups.



AlexNet (Link)

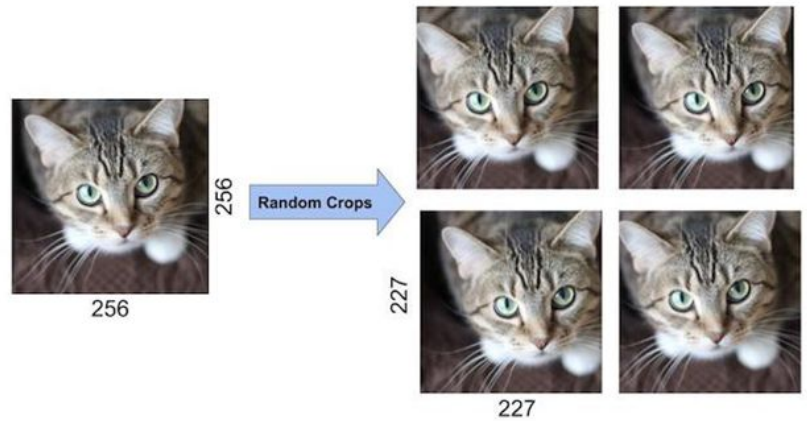


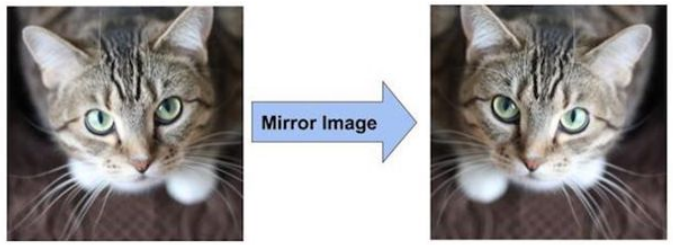
- Alex Krizhevsky (NeurIPS 2012)'s AlexNet architecture is shown here.
- The paper has 185,000 citations.
- ReLU activation is used in every hidden layer. These train much faster and are more expressive than logistic units.

Credit:
https://medium.com/data-science/advanced-topics-in-deep-convolutional-neural-networks-71ef1190522d

AlexNet's Tricks to Improve Generalization

- Train on random 224 × 224 patches from the 256 × 256 images to get more data. Also use left-right mirroring.
  - At test time, combine the opinions from ten different patches: The four 224×224 corner patches plus the central 224×224 patch plus the reflections of those five.



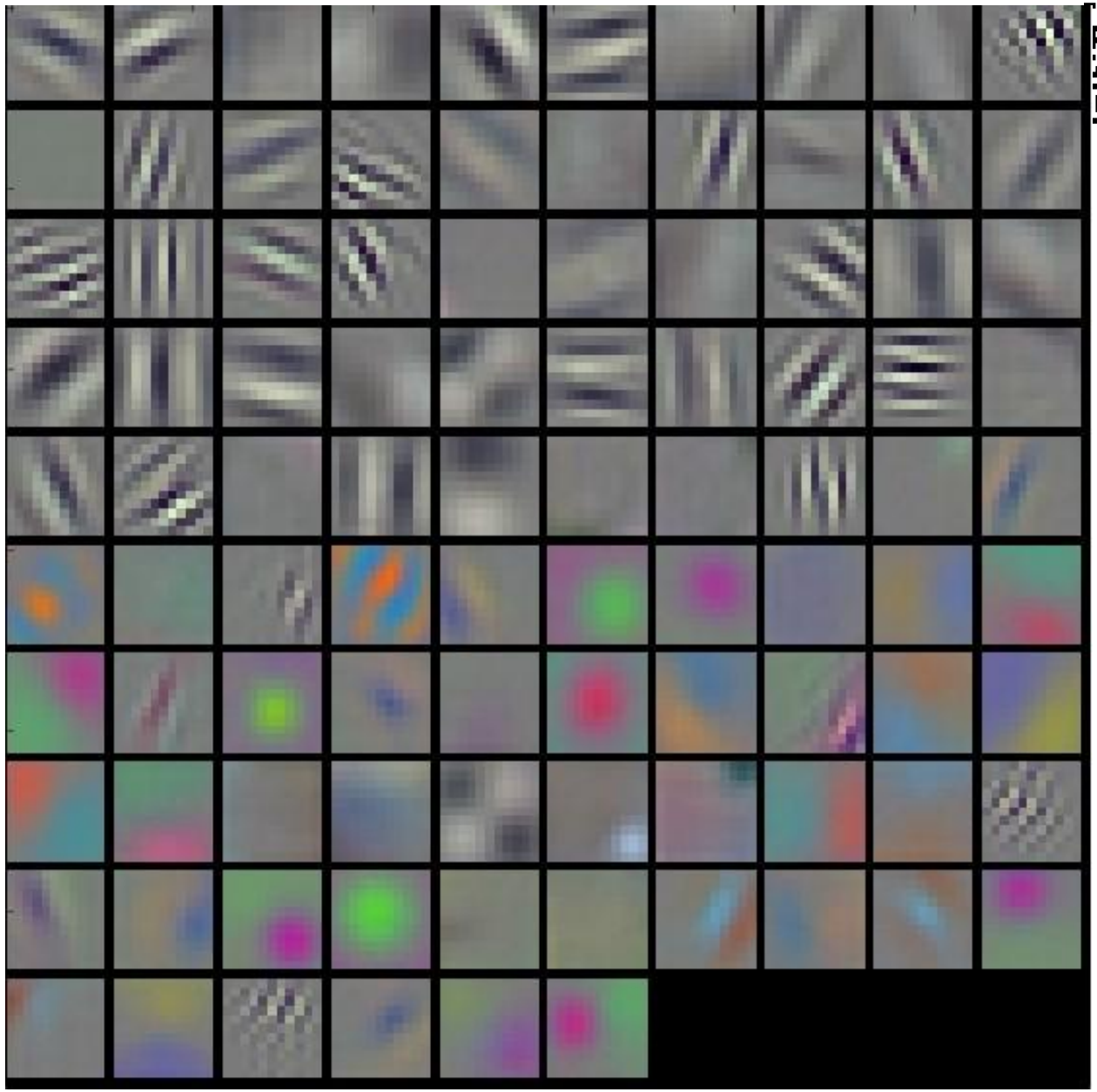


Credit: Geofrey Hinton, University of Toronto

AlexNet's First Convolution Layer

96 convolutional kernels of size 11×11×3 learned by the first convolutional layer on the 224×224×3 input images.

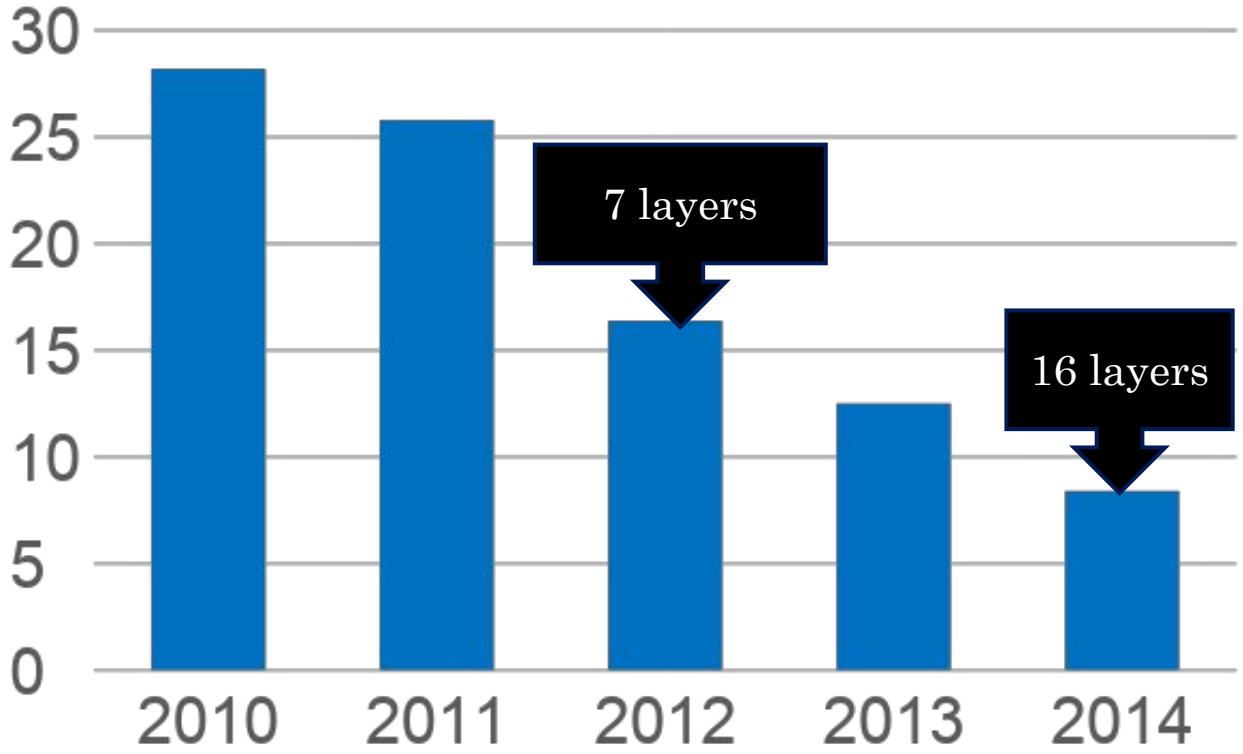
The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2.



3970279

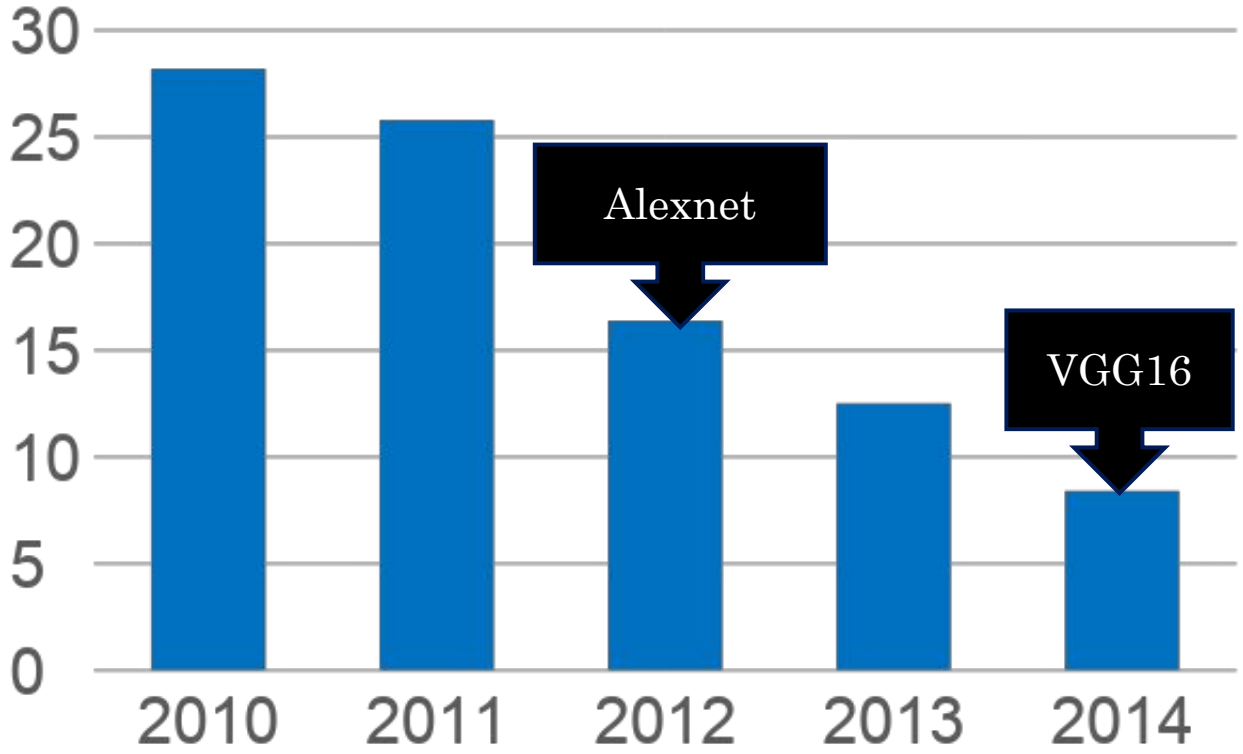
Deeper Is Better

3970279



Deeper Is Better

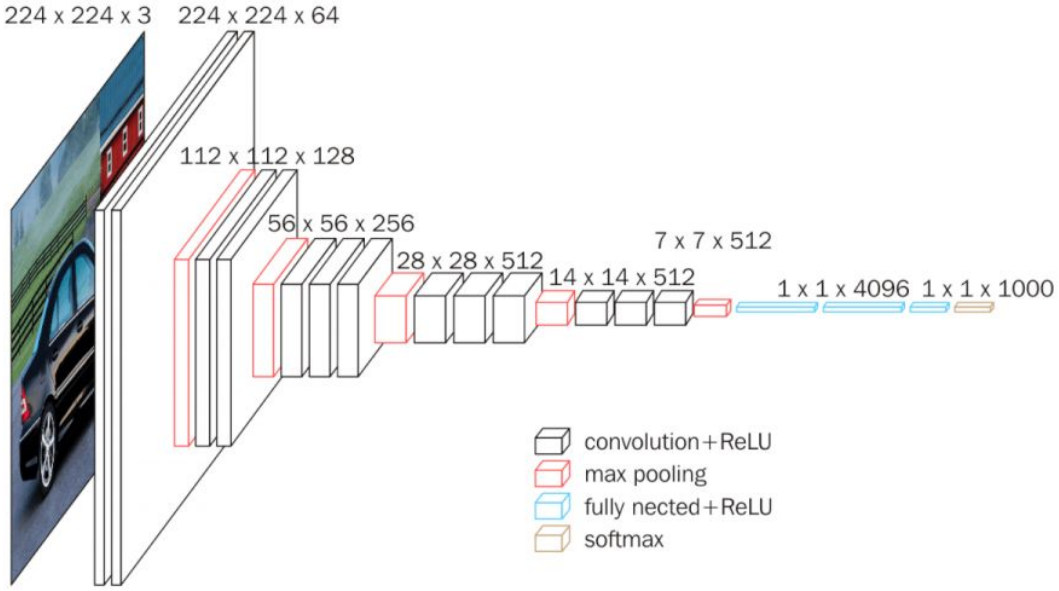
3970279



The VGG pattern

3970279

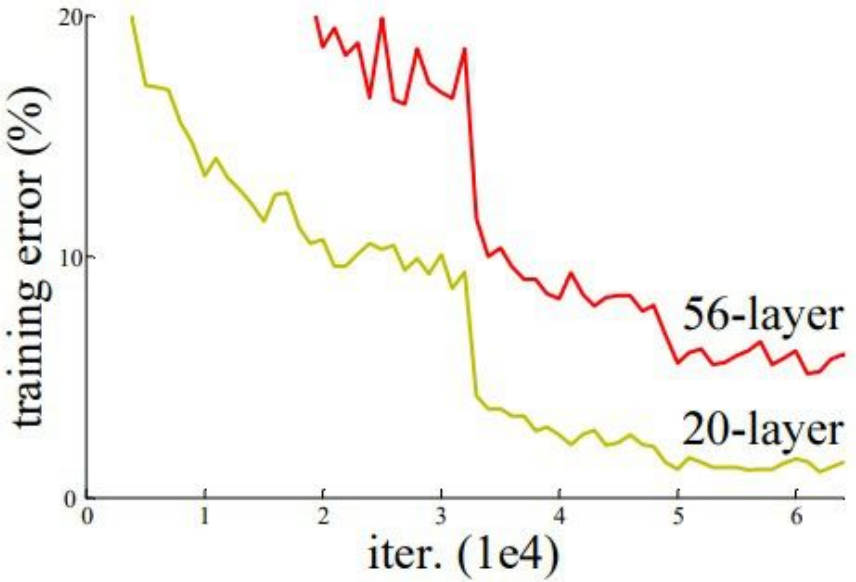
- Every convolution is 3×3, padded by 1, followed by ReLU.
- ConvNet is divided into "stages"
  - Layers within a stage: no subsampling
  - Subsampling by 2 at the end of each stage
- Every subsampling → double the number of channels

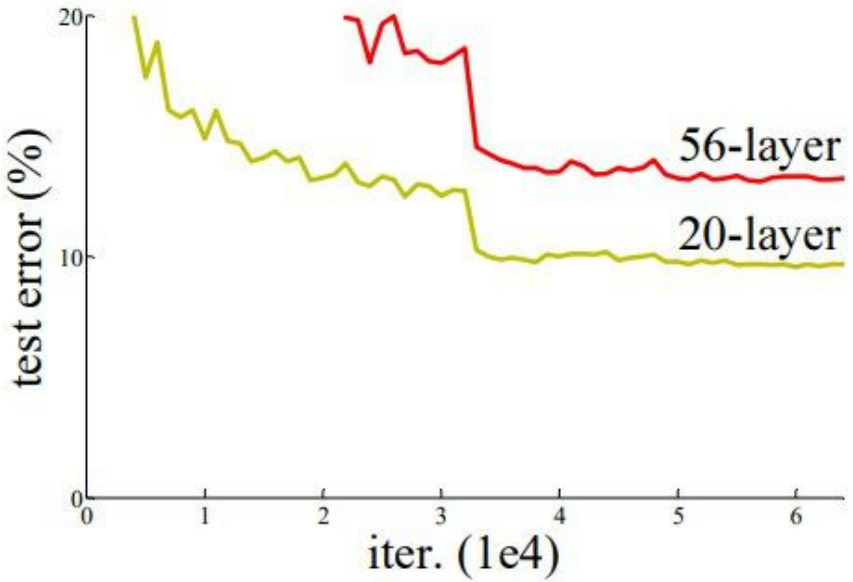


Credit: https://neurohive.io/en/popular-networks/vgg16/

Can We Go Deeper?

3970279





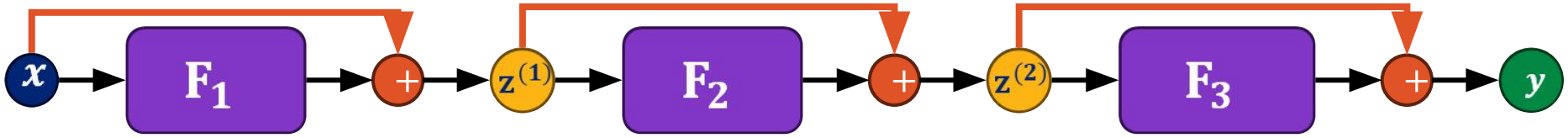
- Vanishing / exploding gradients

$$\frac{\partial z}{\partial z_i} = \frac{\partial z}{\partial z_{n-1}} \frac{\partial z_{n-1}}{\partial z_{n-2}} \cdots \frac{\partial z_{i+1}}{\partial z_i}$$

- Gradient for i-th layer depends on all subsequent layers
  - But subsequent layers are initially random

Residual Connection Fix - Reminder

Residual networks enable the training of very deep networks (e.g., hundreds of layers) by introducing **residual connections** (skip connections) **around blocks** (sub-networks $\mathbf{F}_l$).



We can express a residual block as:

$$\mathbf{z}^{(l)} = \mathrm{F}_l\left(\mathbf{z}^{(l-1)}\right) + \mathbf{z}^{(l-1)}$$

Example:
$\mathrm{F}_l(\mathbf{z}) = W^{(l)}\,\mathbf{ReLU}(\mathbf{z})$

The term **residual** comes from the fact that F_l is modeling the **residual** between the **output** and the **input** of the layer:

$$\mathrm{F}_l\left(\mathbf{z}^{(l-1)}\right) = \mathbf{z}^{(l)} - \mathbf{z}^{(l-1)}$$

ResNet Pattern

- Decrease resolution in first layer
  - Reduces memory consumption
- Divide into stages
  - Maintain resolution, channels in each stage
  - Halve resolution, double channels between stages
- Divide each stage into residual blocks
- At the end, compute average value of each channel to feed linear classifier

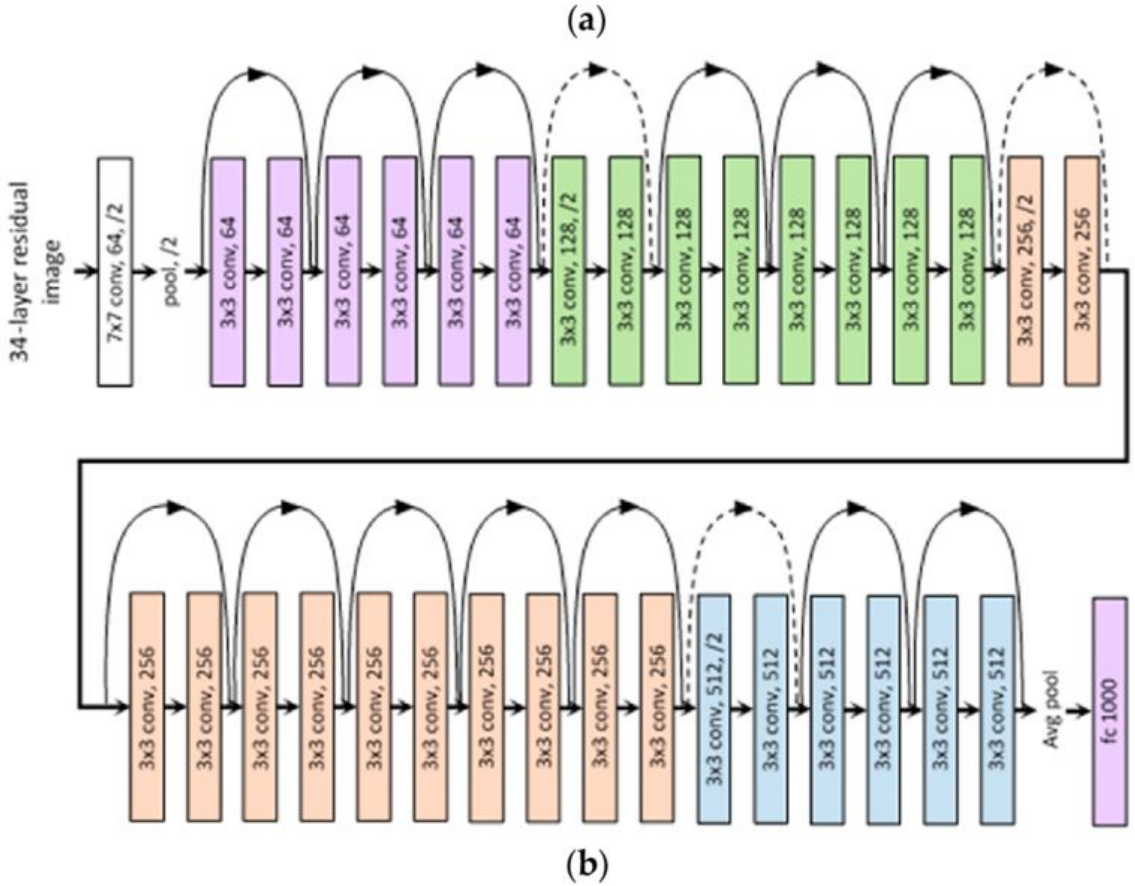
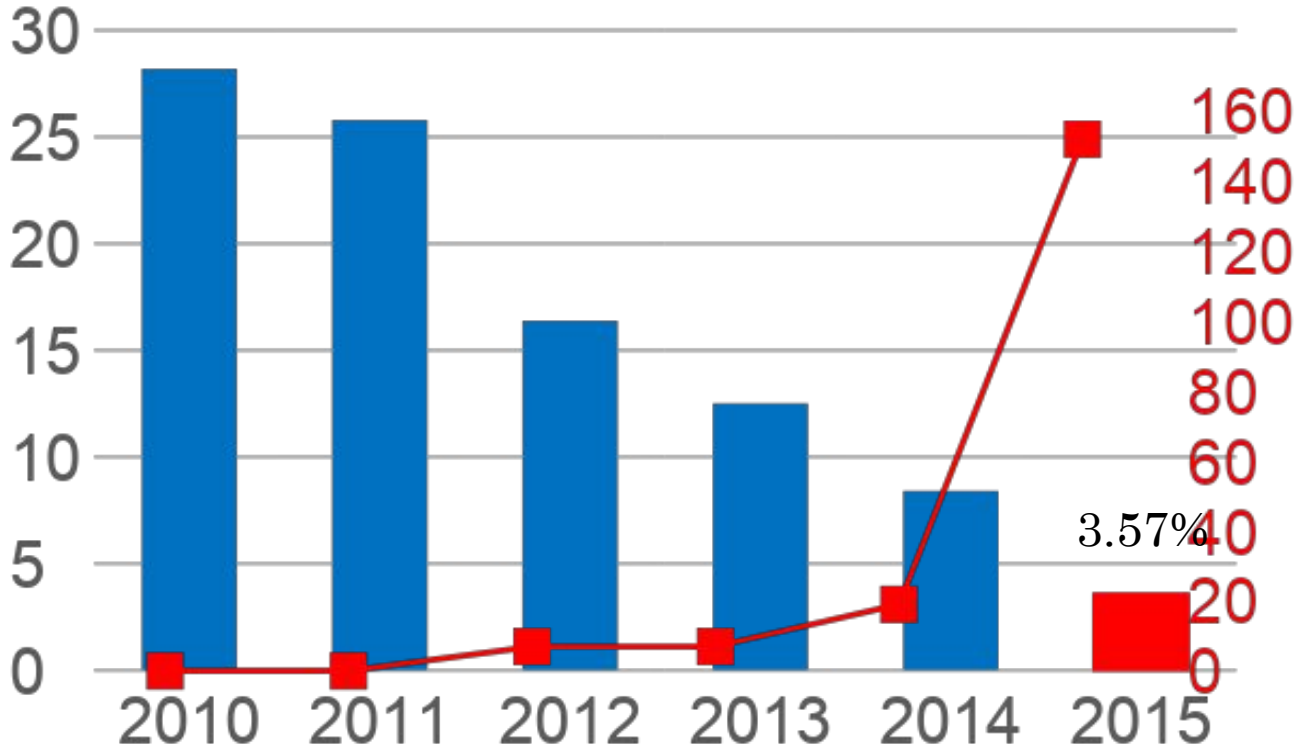


Figure from:
https://medium.com/@siddheshb008/resnet-architecture-explained-47309ea9283d

Putting It All Together – ResNet152

3970279



Transfer Learning

- Recap of CNN Design
- CNN History
- Transfer Learning
- Autoencoders

3970279

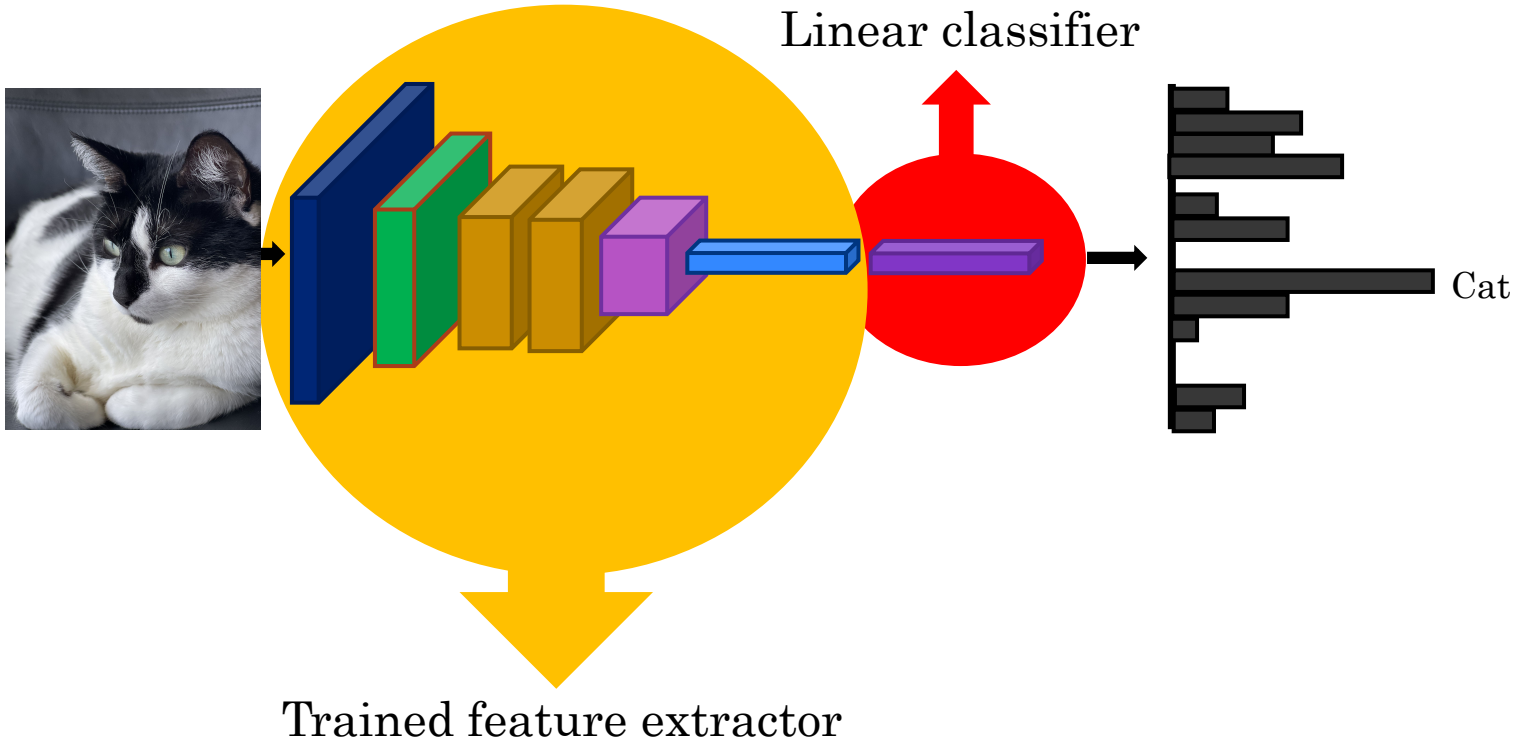
Transfer Learning

3970279

- A **shortcut** in training neural networks for recognition tasks.
- The idea is to start with a fully-trained image recognition neural network, off-the-shelf with trained weights.
- We can **re-purpose** the trained network for our particular recognition task.
- What was learned by the neural network in its early layers are useful features in recognizing various things in images.
- Most of the famous models (pre-trained or not pre-trained) exists on TorchVision: https://docs.pytorch.org/vision/main/models.html

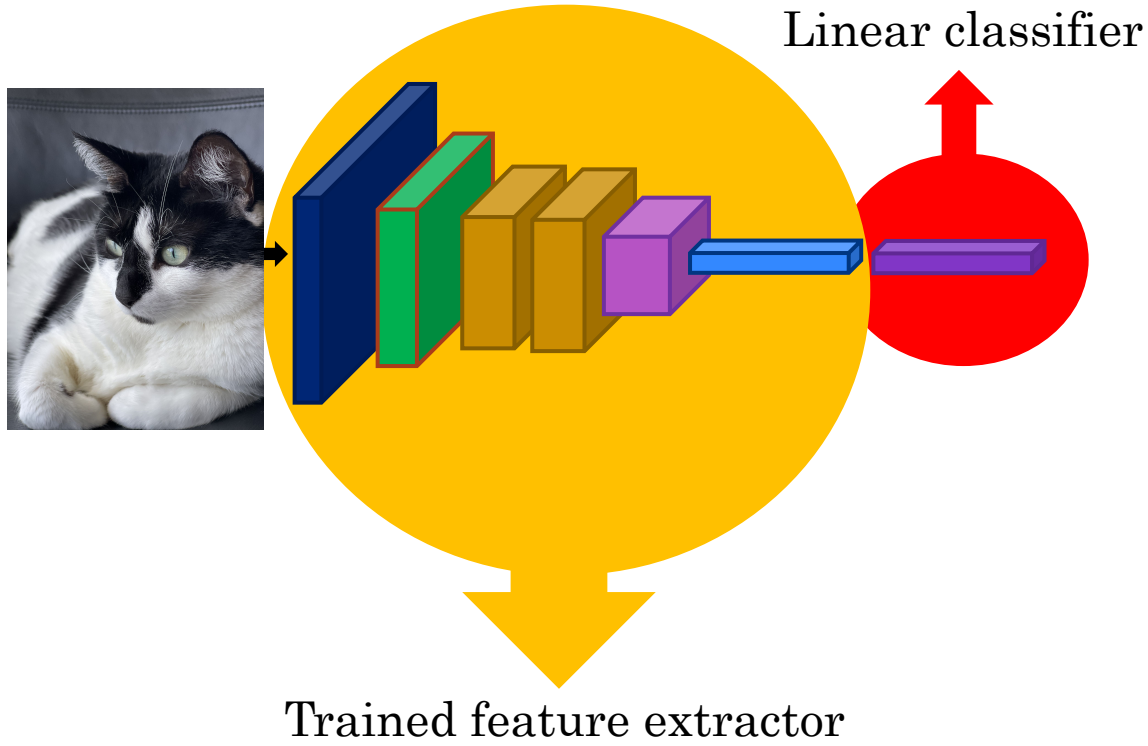
Transfer Learning with CNN

3970279

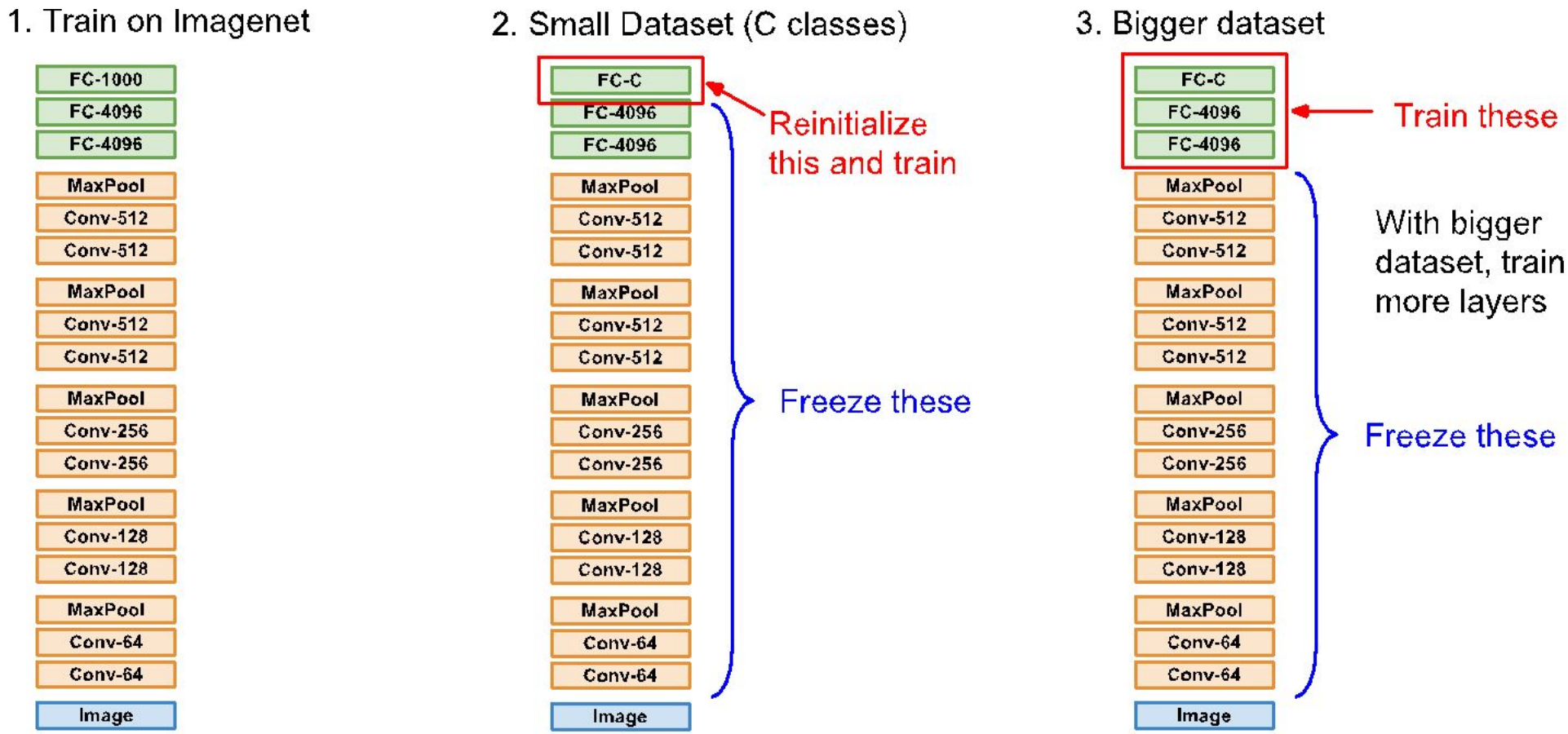


Transfer Learning with CNN

- What do we do for a new image classification problem?
- Key idea:
  - ***Freeze*** parameters in feature extractor
  - ***Re-train*** classifier

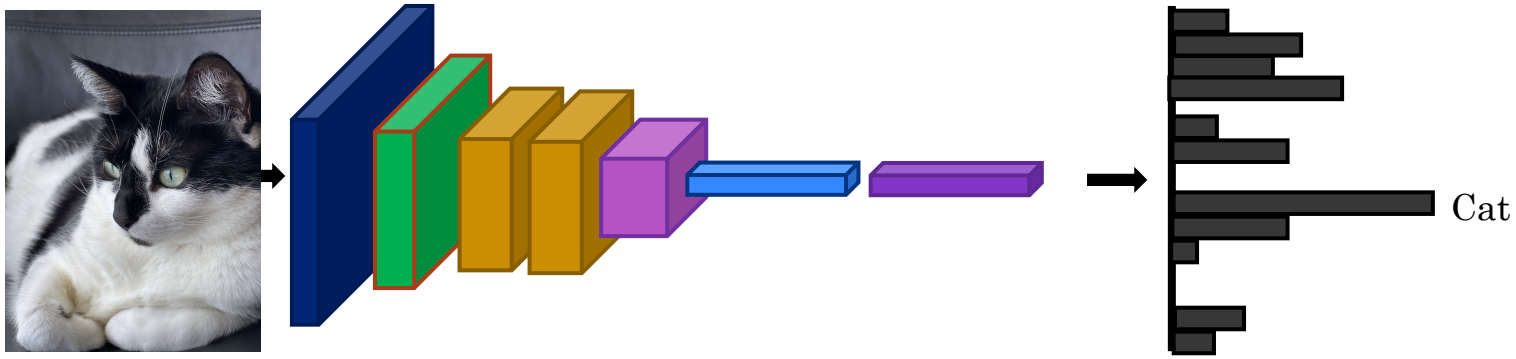


Transfer Learning With CNN

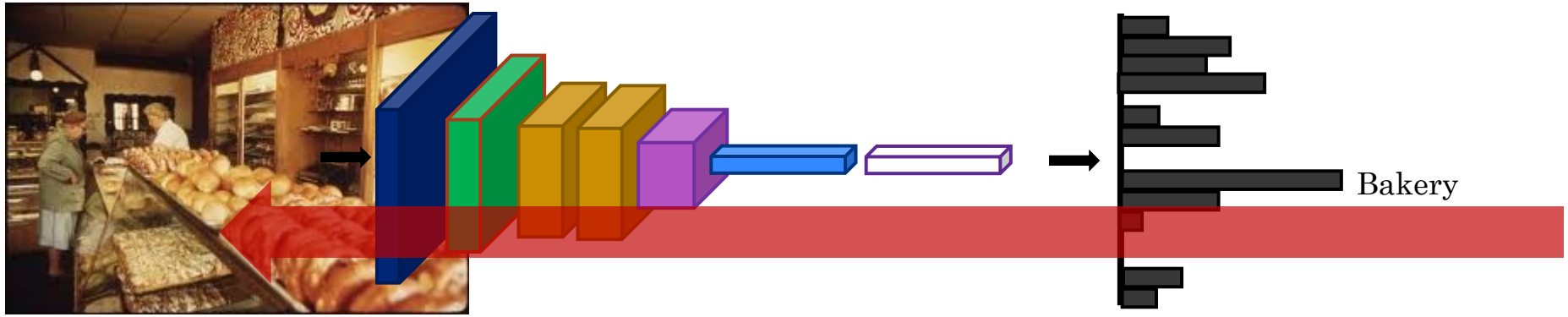


Fine-tuning

3970279



Fine-tuning



Initialize with pre-trained, then train with low learning rate

You have a small labeled dataset for a new classification task. What's the best starting strategy with a pretrained CNN?

Demo

3970279

```
==========================================================================================================================
Layer (type:depth-idx)                   Input Shape               Output Shape              Param #                   Trainable
==========================================================================================================================
VGG16Transfer                            [1, 3, 32, 32]            [1, 100]                  --                        Partial
├─Sequential: 1-1                        [1, 3, 32, 32]            [1, 512, 1, 1]            --                        False
│    └─Conv2d: 2-1                       [1, 3, 32, 32]            [1, 64, 32, 32]           (1,792)                   False
│    └─ReLU: 2-2                         [1, 64, 32, 32]           [1, 64, 32, 32]           --                        --
│    └─Conv2d: 2-3                       [1, 64, 32, 32]           [1, 64, 32, 32]           (36,928)                  False
│    └─ReLU: 2-4                         [1, 64, 32, 32]           [1, 64, 32, 32]           --                        --
│    └─MaxPool2d: 2-5                    [1, 64, 32, 32]           [1, 64, 16, 16]           --                        --
.....
│    └─Conv2d: 2-29                      [1, 512, 2, 2]            [1, 512, 2, 2]            (2,359,808)               False
│    └─ReLU: 2-30                        [1, 512, 2, 2]            [1, 512, 2, 2]            --                        --
│    └─MaxPool2d: 2-31                   [1, 512, 2, 2]            [1, 512, 1, 1]            --                        --
├─Flatten: 1-2                           [1, 512, 1, 1]            [1, 512]                  --                        --
├─Linear: 1-3                            [1, 512]                  [1, 200]                  102,600                   True
├─ReLU: 1-4                              [1, 200]                  [1, 200]                  --                        --
├─Linear: 1-5                            [1, 200]                  [1, 100]                  20,100                    True
==========================================================================================================================

Total params: 14,837,388

Trainable params: 122,700

Non-trainable params: 14,714,688

Total mult-adds (M): 313.60

==========================================================================================================================

Input size (MB): 0.01

Forward/backward pass size (MB): 2.21

Params size (MB): 59.35

Estimated Total Size (MB): 61.58

==========================================================================================================================
```

Autoencoders

- Recap of CNN Design
- CNN History
- Transfer Learning
- Autoencoders

3970279

Deeper into Feature Learning: Autoencoders

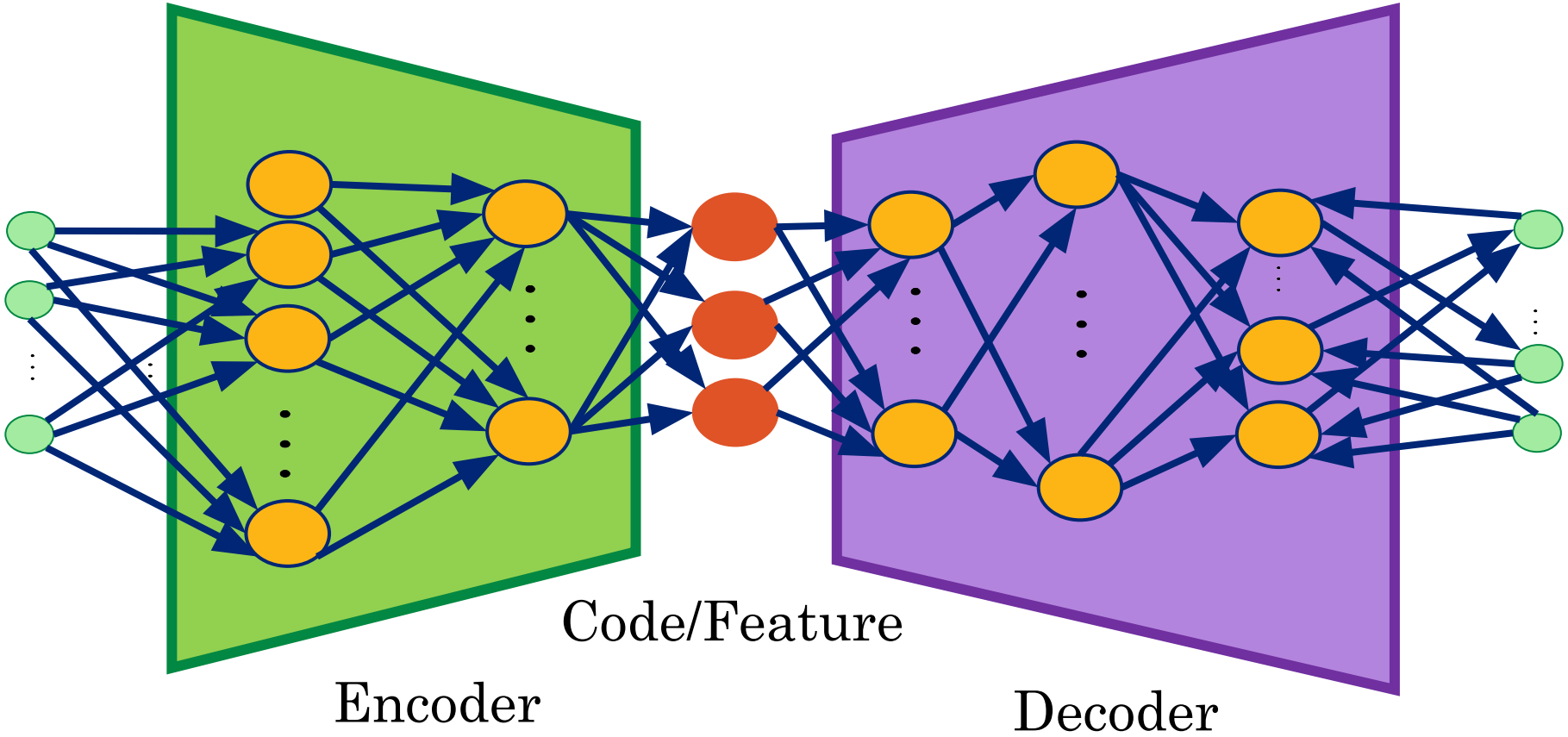
3970279

- Supervised learning uses explicit labels/correct output to train a network.
  - E.g., classification of images.
- Unsupervised learning relies on data only.
  - Key point is to produce a useful embedding of data.
  - The embedding encodes structure such as similarity and some relationships.
  - Still need to define a loss – this is an implicit supervision.

Autoencoders

3970279

- Autoencoders are designed to reproduce their input, especially for images.
  - Key point is to reproduce the input from a learned encoding.



Autoencoder Idea

Given data $\mathbf{x}$ (no labels) we would like to learn the functions f (encoder) and g (decoder) where:

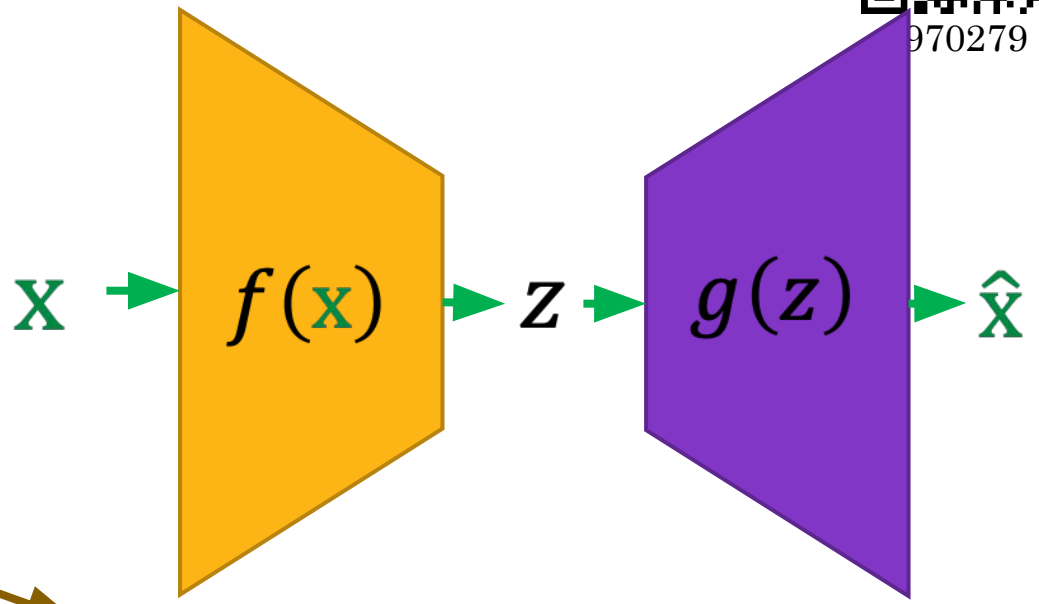
$$f(\mathbf{x}) = s(w\mathbf{x} + b) = z$$

and

$$g(z) = s(w'\mathbf{z} + b') = \hat{\mathbf{x}}$$

$$\text{s.t } h(\mathbf{x}) = g\big(f(\mathbf{x})\big) = \hat{\mathbf{x}}$$

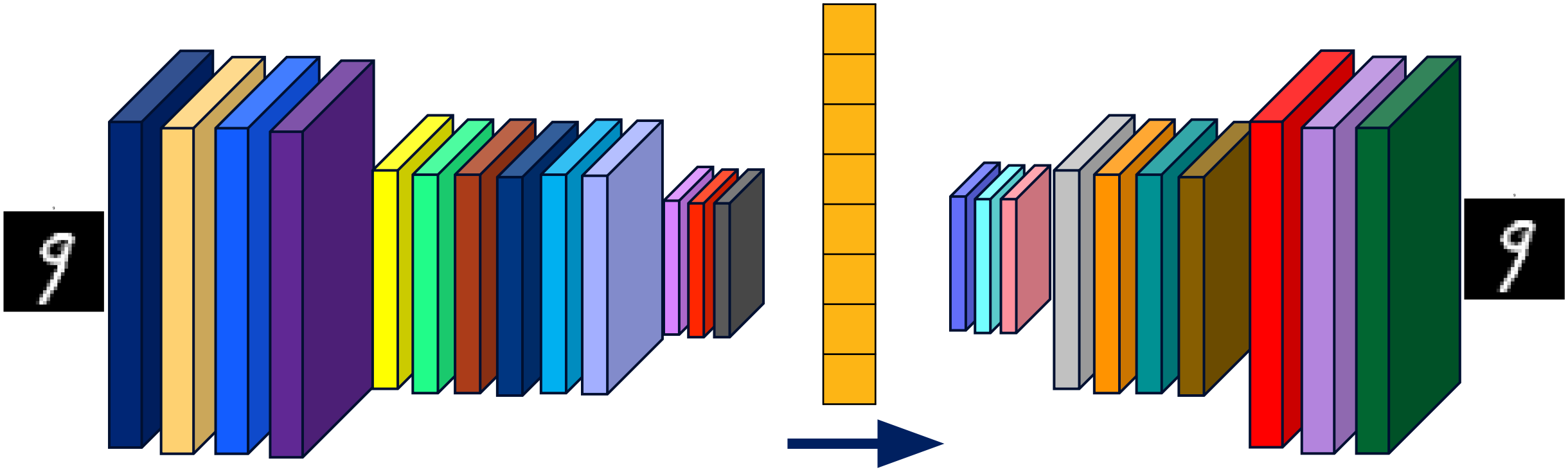
where h is an **approximation** of the identity function.



(z is some **latent** representation or **code** and s is a non-linearity such as the sigmoid)

($\hat{\mathbf{x}}$ is $\mathbf{x}$'s reconstruction)

Convolutional Autoencoder



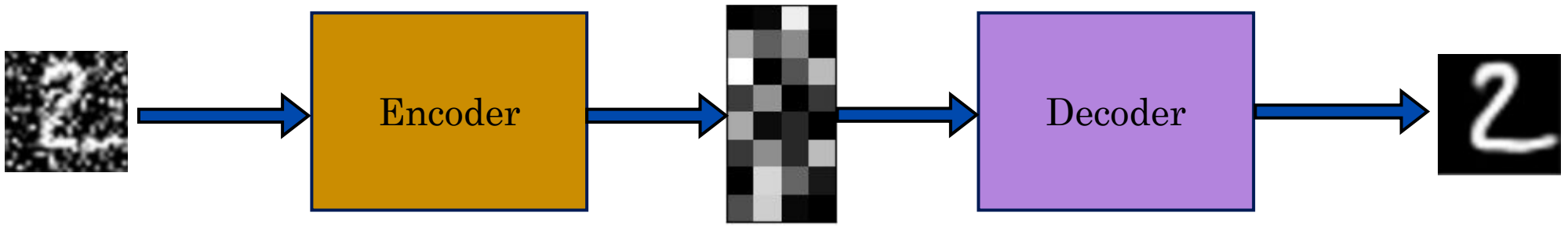
Autoencoders: Applications

- Image colorization: input black and white and train to produce color images.

Autoencoders: Applications

3970279

- Denoising
  - We try to undo the effect of *corruption* process stochastically applied to the input.
  - We still aim to encode the input and to NOT mimic the identity function.



Noisy Input

Latent space representation

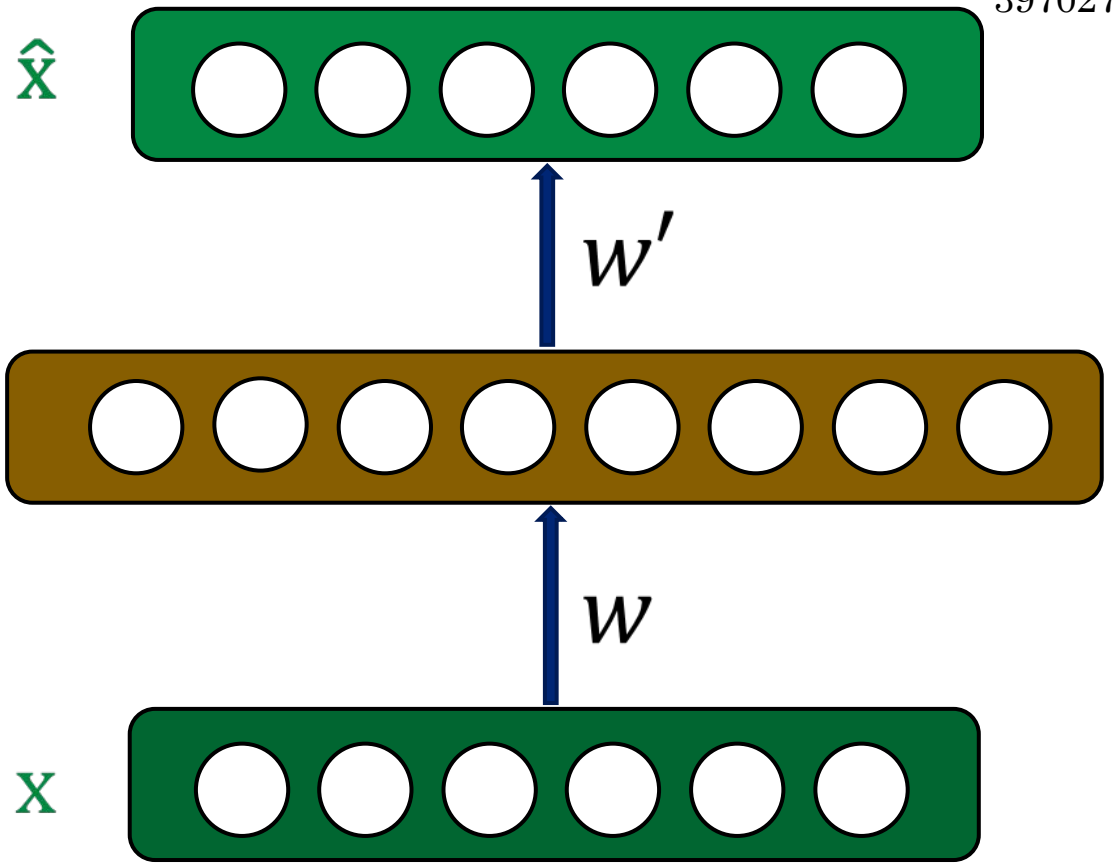
Denoised Input

In a denoising autoencoder, what is the training objective?

Denoising Autoencoders

Idea: A robust representation against noise:

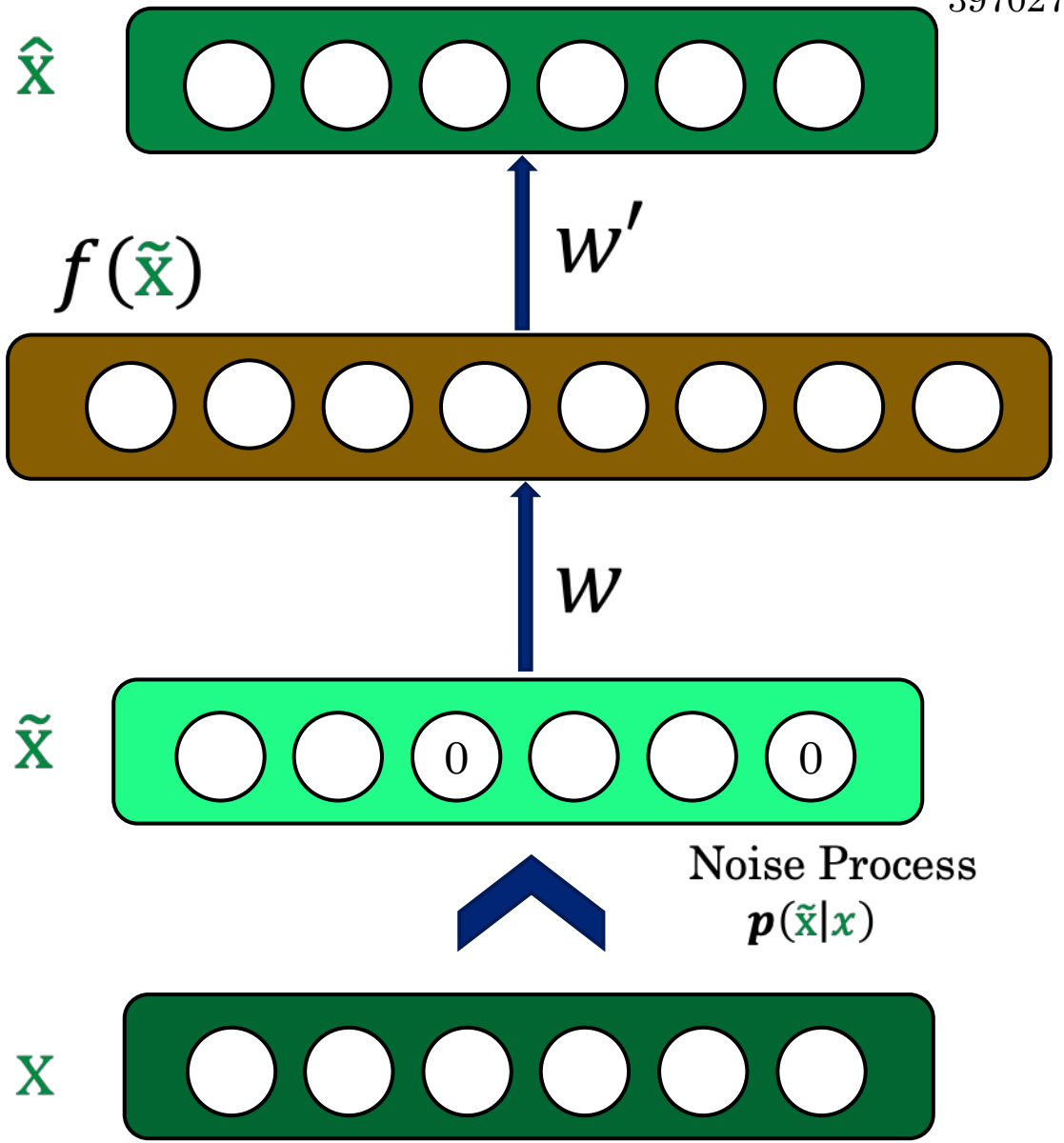
- Random assignment of subset of inputs to 0, with probability v. $f(\mathbf{x})$
- Gaussian additive noise.



(a) (b)

- Reconstruction $\hat{x}$ computed from the corrupted input $\tilde{x}$.
- Loss function compares $\hat{x}$ reconstruction with the noiseless x.

- The autoencoder cannot fully trust each feature of x independently so it must learn from data patterns.
- We are forcing the hidden layer to learn a generalized structure of the data.



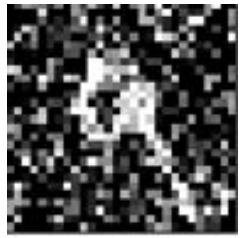
Denoising Autoencoders - Process

3970279

Taken some input x

Apply Noise

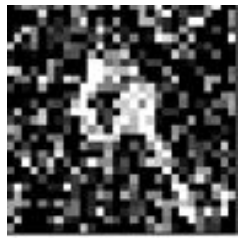
$\tilde{x}$



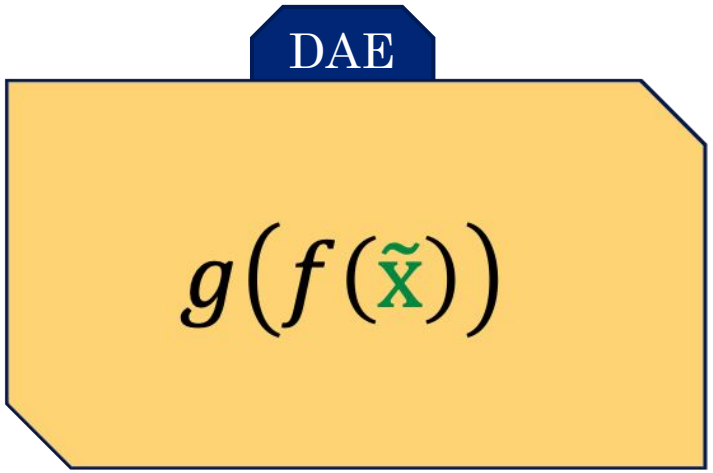
Denoising Autoencoders - Process

3970279

$\tilde{x}$

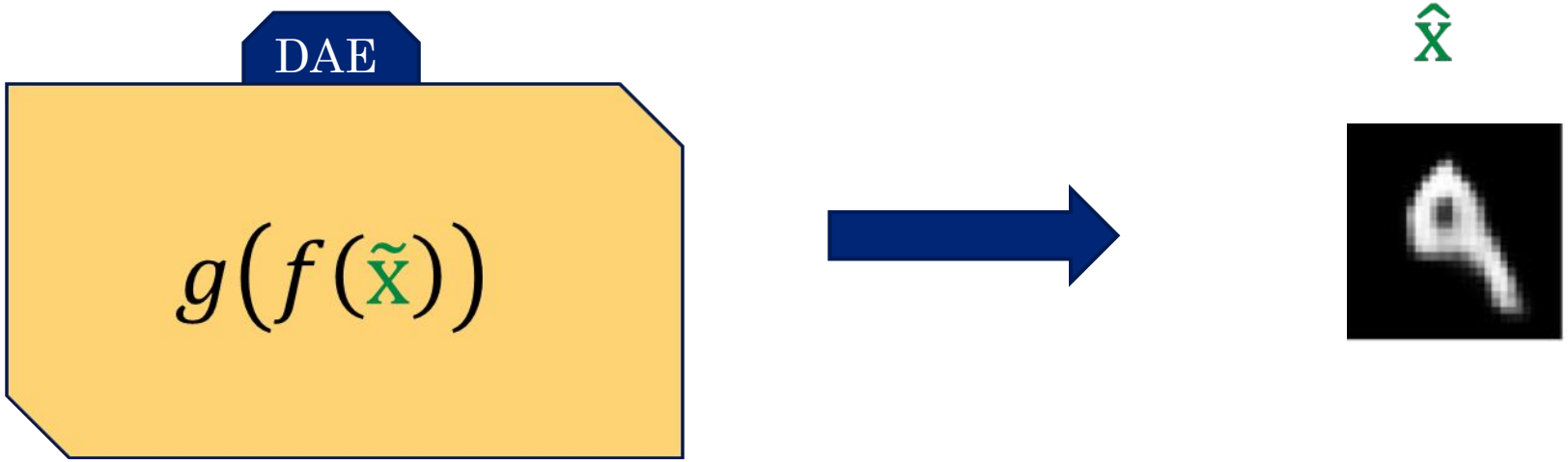


Encode And Decode



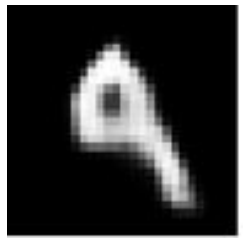
Denoising Autoencoders - Process

3970279



Denoising Autoencoders - Process

$\hat{x}$



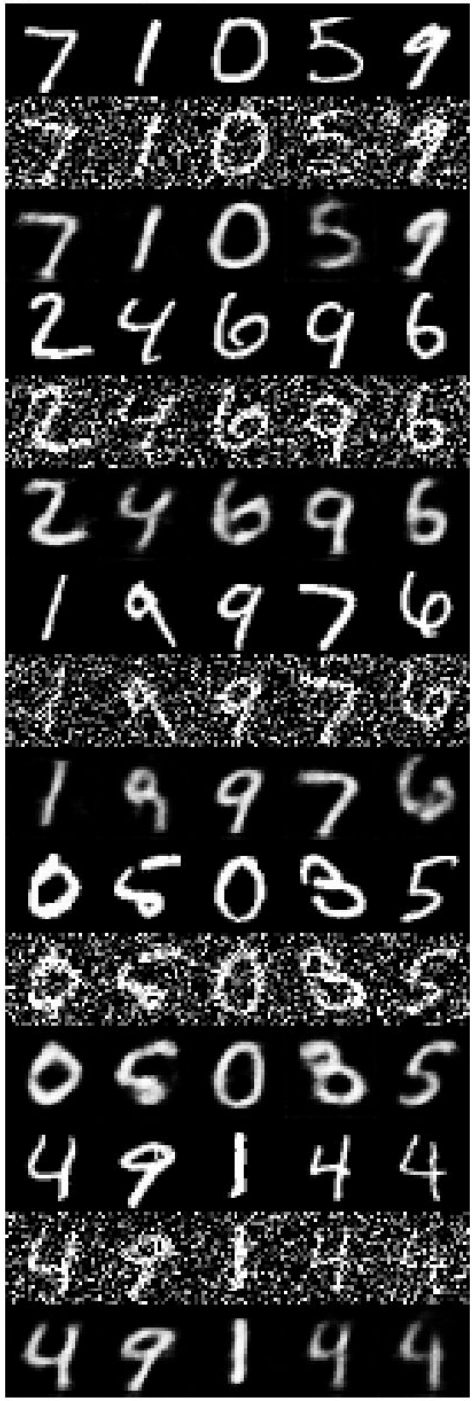
Compare

=

x

Demo

Original images: top rows, Corrupted Input: middle rows, Denoised Output: bottom rows



3970279

Lecture 19

Convolutional Neural Network (Cont.) and Autoencoders

Reading: *Chapter 10* in Bishop Deep Learning Textbook